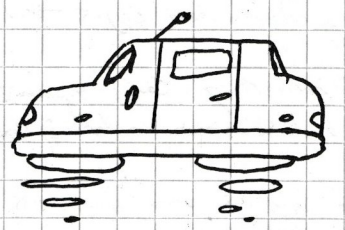
SA

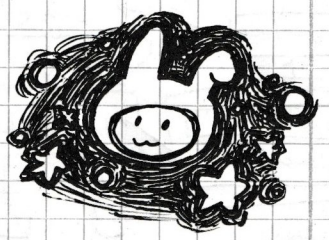
05.12.25

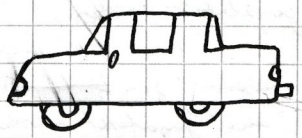
- Klimaanlage
- Sitzheizung
- Rückparkkamera
- E-Auto unschlüssig
- Selbstfahren scheiße
- Fahrassistent ok

25.000€ Budget
10-20 Jahre halten



Haupteinsatz:





Wintertradition:
Blindbaum Christbaum holen

Frau: ästhetisch veranlagt

- letztes Auto: verschrottet in Unfall

① Was willst du erreichen? Was ist dein Ziel? Welches Problem willst du lösen?

② Woran wird die Beschaffung / Architektur festgemacht? → Constraints durch Use Case

③ Constraints und Requirements durch Fachwissen ergänzen → Komplexität erkennen

Compliant Kriterien

→ Nachhaltigkeit des Produkts

☐ Design Patterns – Elements of Reusable OO-Software
Gamma, Helm, Johnson, Vlissides K

☐ Patterns of Enterprise Application Architecture L

☐ Uncle Bob → Clean Code

☐ Enterprise Integration Patterns – Hagge K

L Lesen
K Kaufen

☐ Domain Driven Design – Erick Evans K

☐ Domain Driven Design Kompakt – Vernon

☐ Injection in SQL ausprobieren

☐ Buffer Schneider

☐ XZ Utils Supply Chain Attack

☐ Team Topology

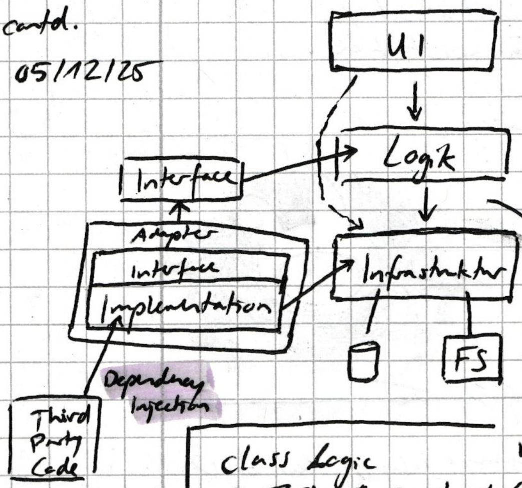
☐ Domain Storytelling – Hofer, Schwentner

Software Qualität / Qualitätsziele

quality.arc42.org

- **Verfügbarkeit** 99,999% im Jahr (bezogen auf Teilsysteme) = System gibt eine sinnvolle Antwort
- **Wartbarkeit** (Wartung, Erweiterbarkeit) → mean time: till deployment, till failure, till recovery
- **Sicherheit**
  - Safety: Sicherheit der Person
  - Security: Sicherheit von Daten und der Applikation
    OWASP → Package Scanner z.B.
- **Usability** Einfachheit der Interaktion, Gebrauchstauglichkeit
  → User Test, Click per Interaction, etc.
- **Performance** Geschwindigkeit, Adaptivität an Randbedingungen (z.B. Laden einer Website bei schlechter Internetverbindung)
  "I/O is where the action is"
  O(n) vs O(n²) vs O(2ⁿ) → erst merkbar bei grossen Datenmässen
- **Testability** Wie viel Test Environment ist nötig? Kleinstmögliche Umgebung?

SA contd.
05/12/25



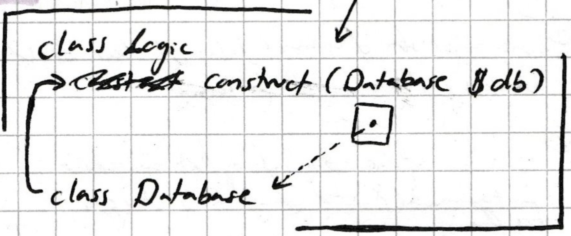
Wartungsproblem bei Logik → Infrastruktur

↳ Third-Party-Code updaten sich eigenständig und können nicht selbst gewartet werden

- bei Update von Database (siehe unten) müssen jegliche Abhängigkeiten geupdatet werden

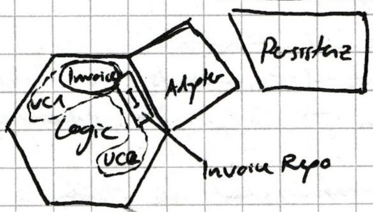
↳ Logik (was eig unabhängig von ~~Database~~ Infrastruktur ist responsibility-wise) muss auch geupdatet werden

Bsp.



- Um dieses Problem zu lösen, kann man das **Dependency Inversion Principle** benutzen

- **Adapter** Implementierung des Interfaces unter Verwendung des Third Party Codes und angepasst an den eigenen Use Case

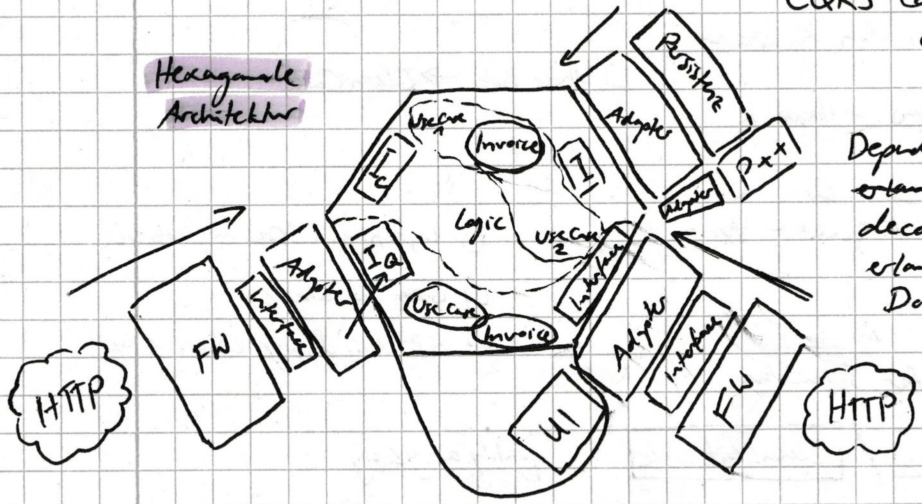


Hexagonale Architektur

User Stories — tools to determine process
Event Stories — 1|2|3|4 → □
Domain Storyt.

CQRS Command Query Responsibility Segregation

Hexagonale Architektur



Onion Architecture (Zwiebelschichten)

Dependency Inversion:
~~erlaubt Implementierung von eigener~~
decoupled Abhängigkeiten,
erlaubt eigene Festlegung von Domäne und wie diese funktioniert
erlaubt Modularität

FW Framework

Allgegenwärtige Sprache (ubiquitous language): im Kontext des Use Case wird eine Sprache benutzt, welche ~~...~~ vorgegeben ist und auf allen Level der Entwicklung vertreten ist, sodass einfache Kommunikation und Verständnis ~~...~~ gegeben ist. → Bounded Kontext

Domain Driven Design

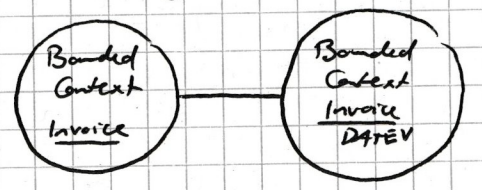
Core Domain (make)
Subdomain → Teilproblem / Teilprozess (buy)
- Supporting
- Generic (must have for system but annoying, best to use ready-to-use)

SA
cont'd
05/12/25

[redacted]

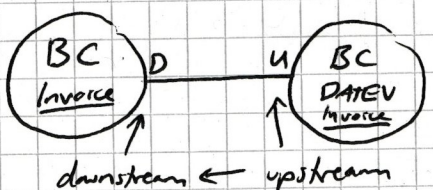
Core: Analyse und Prognose von Energieverbrauch und [redacted]-erzeugung

Sub: Erfassung der Daten, Umsetzung der Optimierung (Steuerung)

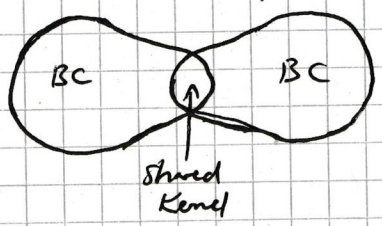


- Big Ball of Mud: nobody knows how exactly it works

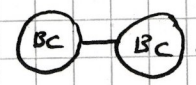
- Customer-Supplier: Supplier erfüllt die Wünsche des Customers



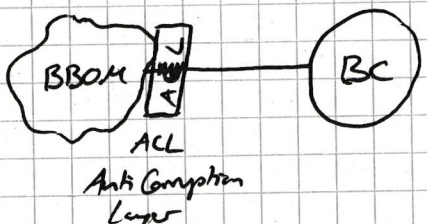
- Conformist: konform mit dem existierenden System, Kopplung



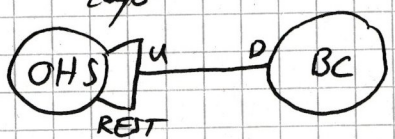
- Shared Kernel: beide BC haben ein SK (Code), den beide bearbeiten und maintainen



- Partnership: gleichberechtigt, gemeinsame Problemlösung



- Anti-Corruption: Bau einer stabilen mentalen Schnittstelle, die das eigene BC vor dem anderen schützt



- Open Host Service (OHS): REST API

- Published Language: Sprachspezifikation

- Separate Ways

[redacted]

☐ SMART (Title)

Context Map Cheat Sheet

Context Map Patterns

Open / Host Service

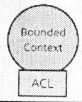
A Bounded Context offers a defined set of services that expose functionality for other systems. Any downstream system can then implement their own integration. This is especially useful for integration requirements with many other systems. Example: public APIs.

Conformist

The downstream team conforms to the model of the upstream team. There is no translation of models. Couples the Conformist's domain model to another bounded context's model.

Anticorruption Layer

The anticorruption layer is a layer that isolates a client's model from another system's model by translation. Only couples the integration layer (or adapter) to another bounded context's model but not the domain model itself.

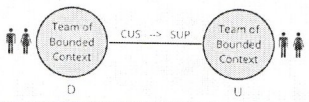


Shared Kernel

Two teams share a subset of the domain model including code and maybe the database. Typical examples: shared JARs, DLLs or a shared database schema. Teams with a Shared Kernel are often mutually dependent and should form a Partnership.

Customer / Supplier

There is a customer / supplier relationship between teams. The downstream team is considered to be the customer. Downstream requirements factor into upstream planning. Therefore, the downstream team gains some influence over the priorities and tasks of the upstream team.

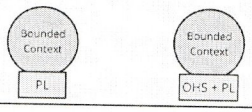


Partnership

Partnership is a cooperative relationship between two teams. These teams establish a process for coordinated planning of development and joint management of integration.

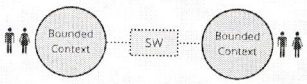
Published Language

A Published Language is a well documented shared language between Bounded Contexts which can translate in and out from that language. Published Language is often combined with Open Host Service.Typical examples are iCalendar or vCard.



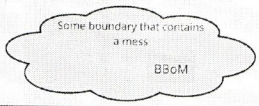
Separate Ways

Bounded Contexts and their corresponding teams have no connections because integration is sometimes too expensive or it takes very long to implement. The teams chose to go separate ways in order to focus on their specific solutions.



Big Ball Of Mud

A (part of a) system which is a mess by having mixed models and inconsistent boundaries. Don't let this lousy model propagate into the other Bounded Contexts. Big Ball Of Mud is a demarcation of a bad model or system quality.



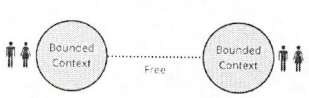
Team Relationships

Mutually Dependent

Two software artifacts or systems in two bounded contexts need to be delivered together to be successful and work. There is often a close, reciprocal link between data and functions between the two systems.

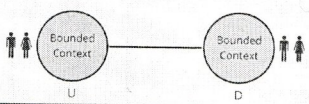
Free

Changes in one bounded context do not influence success or failure in other bounded contexts. There is, therefore, no organizational or technical link of any kind between the teams.



Upstream / Downstream

Actions of an upstream team will influence the downstream counterpart while the opposite might not be true. This influence can apply to code but also on less technical factors such as schedule or responsiveness to external requests.



SA

12/12/25

Client: I want to build a spaceship and go into space

good: admitting when smth is not part of one's domain → calling a specialist

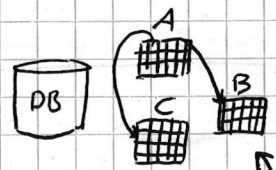
bad: asked where he wanted to go but core problem/need is actually that he thought Elon Musk and his spaceships are cool
→ did not ask enough about core need/want

> 5 Whys: dig deeper and get to the core problem/need by repeatedly asking why

ORM Object-Relational Mapping

SQL → eliminate need to write queries

OOP

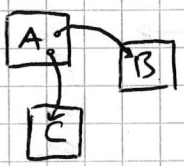


TABLE

ROW

ROW DATA GATEWAY

Object based on ROW



isomorph

Problem: changes in the table need to be reflected back into the object and vice versa

Object

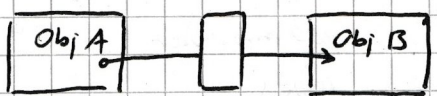
save()

save(): reflect new info back to table

Active Record → refers to ROW

eager loading: loading it immediately

Lazy loading: loading it when needed



example: Library with 5000 books

Lazy Loading Proxy

creates 5001 queries → does not work

"ORM ≠ is the Vietnam of our industry"

n+1 Problem: works on small scale but not scaleable

TABLE

A

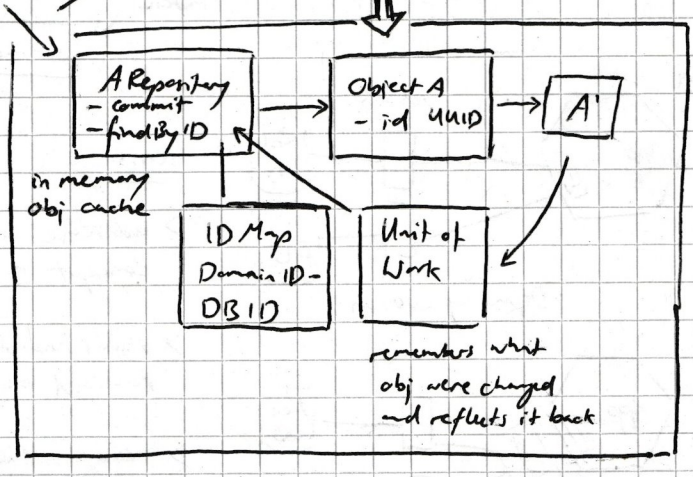
ID = int

Data Mapper

~~Object A~~

how to reflect back?

- does not solve n+1 problem
- does not solve concurrency problem
- ORM is not solution to everything but enough depending on use case



SA

cntd.

12/12/25

- need to always look at and dissect problem to find optimal solution

NoSQL (Not only)

Key-Value Store (hash map / associated array)

key => value

"key": "value"

bib_book: ...

→ want to know what books the bib has?

→ back to n+1 problem

Distributed Key-Value Store DKV

Document DB / Collection Based

Search Engine

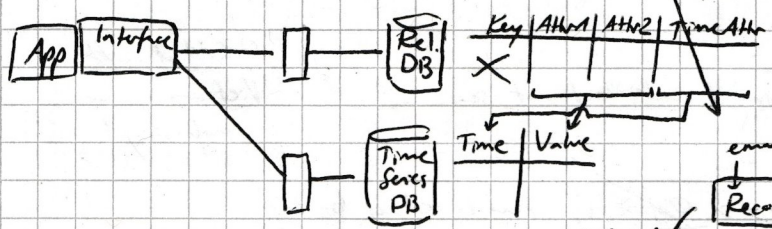
A: foo bar baz	→	index:
B: bla bla bla		foo A, C
C: foo bla		bar A
		baz A
		bla B, C
		⇨ reverse index

Polyglot Persistence

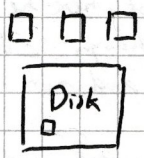
Graph DB Triple Stores → Wikipedia mit wikiData (Wissensdatenbank)

Time-series DB Column-oriented DB

Übung: Migration from Rel. DB to Time Series DB

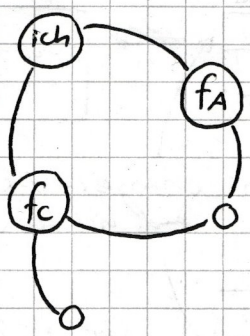


SW



> no matter what pattern, essentially we want to write data to the disk

> but if we don't care about persistency, then we can process data really fast

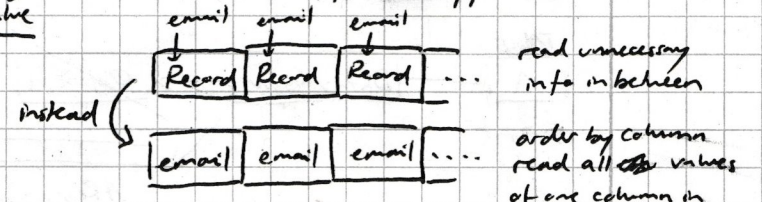


100 mio User

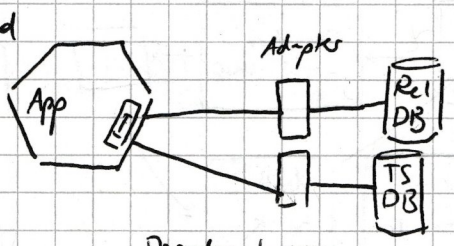
USER

id	email	...
⋮		

> get email of all users, what happens?

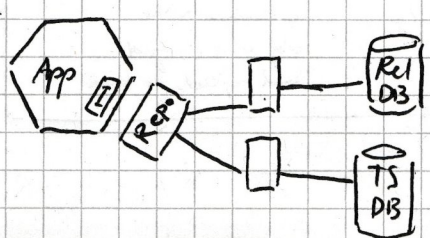


Read



Dependency Inversion

Write



learnings:

> use case necessary to add depth to concept (possible)

> visual should be clean and concise to avoid misunderstandings later

> read/write case should be considered separately

> no preemptive optimization/considerations, only do what is needed

S4
cntd.
12/12/25

> abstract modelling of domain events, divorced from technical implementation (abstract)

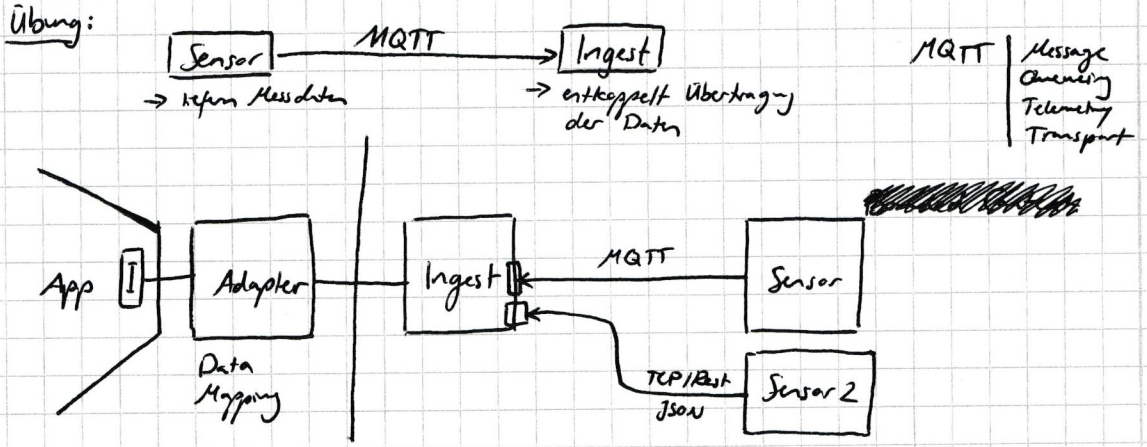
ex.: [Temperature measured] → [Ø Temperature calculated] → [decision made]

>not important where temp measurements are saved for this process

Conway's Law Organizations, who design systems, are constrained to produce designs which are copies of the communication structures of these organizations.

> simplicity over complexity both in the architecture and communication
→ breaking down problems (divide & conquer)
→ only do what needs to be done ~~at~~ at the moment (Segae)
~~The~~ architecture should meet ~~our~~ current needs

Übung:



>if unsure about component (what does this even do? What is it for?), leave it out and check if it works without

>testing can be done ~~in any context~~ within any boundary: in App with generated data set
in Adapter with sensor test data
at Sensor with a simulation

→ if done right, then each bounded context is testable on its own **TESTABILITY**

Dissecting Systems Criteria: > physical constraints (server)

> need for buffer ~~(Sensibilität, Unterbrechung)~~

> scalability, availability (quality goals)

Üb: Bausteen-Diagramm, Aufbrechung des Monoliths
- Datenbanken sollten immer Teil der Blöcke (Baustein) sein

(Aufg.: Projekt wurde ~~als~~ als Monolith betrachtet und sollte in Einzelkomponente anhand von den davon Kriterien aufgebrochen werden)

Was ist Software Architektur? > kontinuierlich
> erweiterbar

> Disziplin der Konzeption eines Software Systems, samt dessen Wartung und Lebenszyklus

> basiert auf needs und requirements des Kunden → lösungsorientiert

> Aufstellung des Systems, team basiert, Domain Analyse

> Zusmspiel von Komponenten, ihren Verantwortlichkeiten, Schnittstellen und Abhängigkeiten (Interaktion)
→ Anforderungen des Systems erfüllen

SA
cont'd.

12/12/25

- [ ] Was ist MVC?
  Welches Problem löst es?
  → Pattern erklären können

- [ ] State of src 42
  15min, jeweils 5
  (keine extra Folien)